

AWS Infrastructure Deployment using Terraform

Objective

Build a complete AWS infrastructure using Terraform. You'll create a secure, scalable setup with multiple availability zones, proper networking, database, storage, and monitoring. This will give you hands-on experience with Infrastructure as Code and real-world AWS architecture.

What You Need to Build

Networking:

- Create a VPC with CIDR 10.0.0.0/16
- 2 Public subnets (one in each AZ) for web servers
- 2 Private subnets (one in each AZ) for application & database
- Internet Gateway for public subnet internet access
- NAT Gateway for private subnet outbound traffic
- Route tables configured and associated properly

Compute (EC2):

- Bastion Host in public subnet (for SSH access)
- Application Server in private subnet
- Security groups with proper inbound/outbound rules
- Use t3.micro or t3.small instance types

Database & Storage:

- RDS MySQL in private subnets with Multi-AZ
- S3 bucket with versioning and encryption enabled
- Lifecycle policies for S3 objects

IAM & Security:

- Create IAM roles for EC2 instances
- Create S3 access policy
- Use security groups with least privilege access
- Enable encryption on S3

Monitoring & DNS:

- CloudWatch alarms for EC2 CPU
- CloudWatch dashboard
- Route53 hosted zone with sample DNS record

Terraform Code Requirements

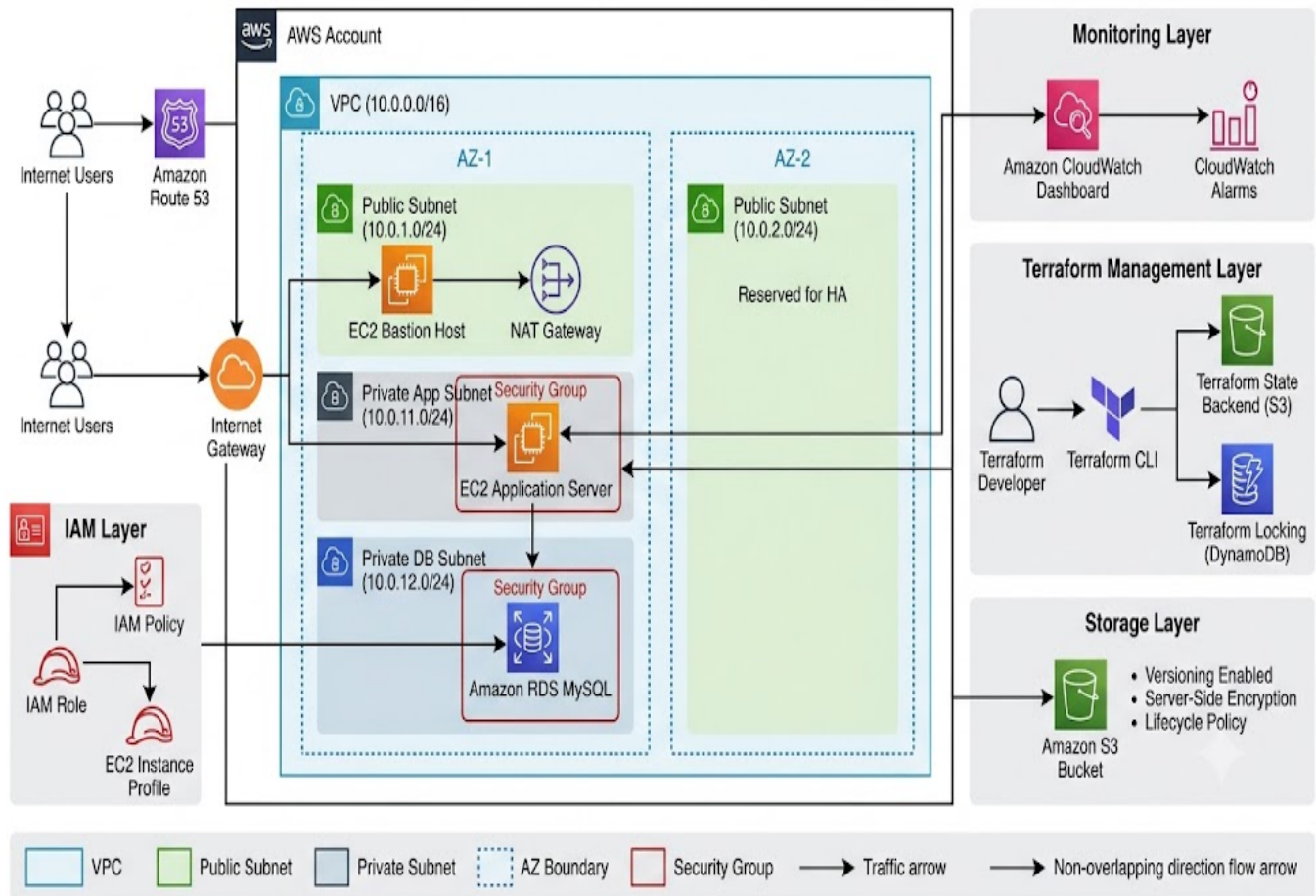
- Use modules (VPC, EC2, RDS, Monitoring)
- Separate files: variables.tf, outputs.tf, main.tf, provider.tf
- Use terraform.tfvars for environment values
- Remote backend in S3 with DynamoDB locking
- Use for_each, count, locals, and dynamic blocks
- Create dev and prod workspaces
- Follow consistent naming conventions
- Add comments to explain the code

What to Submit

- Terraform code (organized in proper structure)
- README.md with setup and deployment steps
- Screenshots of deployed resources
- AWS cost estimation
- Terraform state backend configuration
- Architecture diagram

Architecture Diagram

Production-Ready AWS Infrastructure using Terraform



Architecture Overview

The diagram shows how all components connect together. Users reach your application through Route53 DNS, which points to the Internet Gateway. The VPC spans 2 availability zones for high availability. Bastion Host sits in public subnet for secure SSH access. Application server in private subnet handles business logic. RDS database runs in private subnets with Multi-AZ. S3 stores your files securely. CloudWatch monitors everything, and Terraform manages all infrastructure through code.

Key Points to Remember

- Public subnets = direct internet access (web servers, bastion)
- Private subnets = no direct internet (app server, database)
- NAT Gateway allows private subnets to reach internet for updates
- Security groups = firewall rules for each resource
- RDS Multi-AZ = automatic failover if primary fails
- Terraform modules = reusable, maintainable code
- S3 state locking prevents concurrent modifications